

Practical Practice

ID607001: Introductory Application Development Concepts

Assessment Overview

In this individual assessment, you will write unit tests for an existing backend application - a **Quiz API** built with Express and Prisma. The application integrates with the [Open Trivia Database \(OpenTDB\)](#) API and supports two roles: **creator** (who creates and manages quizzes) and **player** (who plays quizzes).

You will not build the application from scratch. Instead, you will clone the provided starter repository, read and understand the existing code, and write unit tests that verify the correctness of its controllers and middleware using **Mocha**, **Chai** and **Sinon**.

Application Overview

Domain Description

The application supports two types of users:

- **Creator** - can create and manage quiz categories, quizzes, and questions. Creators can import questions directly from the OpenTDB API into a quiz.
- **Player** - can browse available quizzes, start a game session, submit answers, and view their score history.

Models

Model	Description
User	A registered user with a role of either PLAYER or CREATOR
Category	A topic area for quizzes (e.g. Science, History), created by a creator
Quiz	A quiz belonging to a category, created by a creator
Question	A multiple-choice question belonging to a quiz, with one correct answer and three incorrect answers

Project Structure

```
backend/  
├── .mocharc.json  
├── .c8rc  
├── controllers/  
│   └── auth.js
```

```

├── category.js
├── quiz.js
├── middleware/
│   ├── jwtAuth.js
│   └── rbac.js
├── repositories/
│   ├── category.js
│   └── quiz.js
├── tests/
│   ├── mocks/
│   │   ├── category.mock.js
│   │   └── quiz.mock.js
│   └── unit/
│       ├── 00-auth.test.js
│       ├── 01-category.test.js
│       ├── 02-quiz.test.js
│       ├── 03-jwtAuth.test.js
│       └── 04-rbac.test.js

```

Testing Tools

Library	Role
Mocha	Test runner - organises tests into suites and runs them
Chai	Assertion library - verifies that values match expectations
Sinon	Mocking library - replaces real dependencies with controlled fakes
c8	Code coverage - measures how much code your tests execute

Assessment Requirements - Unit Tests

Setup (not marked)

1. Clone the provided starter repository and open it in Visual Studio Code.
2. Install dependencies: `npm install`.
3. Confirm the following scripts are present in `package.json`:

```

"test": "mocha",
"test:coverage": "c8 mocha",
"test:coverage:report": "c8 report --reporter=html"

```

4. Confirm `.mocharc.json` exists in `backend/` with the following content:

```

{
  "spec": "tests/unit/**/*.test.js",
  "timeout": 10000,

```

```
"exit": true
}
```

5. Read through the `controllers/`, `middleware/` and `repositories/` directories before writing any tests.

Shared Mock Helpers

Before writing any test suites, create shared mock helpers in the `tests/mocks/` directory. Each file must own the helpers for one domain and be imported into the corresponding test file rather than redefining the helpers each time.

- Create a mock helper file for each model that has a repository (`category`, `quiz`). Each file must export a `mockReq`, `mockRes` and a repository stub factory for that domain.
- Each `mockRes` must implement `res.status` as a Sinon stub that returns `res` itself, so that chained calls like `res.status(201).json(...)` work correctly.

Here is the expected shape of each mock file:

```
import sinon from "sinon";
import someRepository from "../../repositories/some.js";

export const mockReq = (body = {}, params = {}, query = {}) => ({
  body,
  params,
  query,
});

export const mockRes = () => {
  const res = {};
  res.status = sinon.stub().returns(res);
  res.json = sinon.stub().returns(res);
  return res;
};

export const stubSomeRepo = () => ({
  create: sinon.stub(someRepository, "create"),
  findAll: sinon.stub(someRepository, "findAll"),
  findById: sinon.stub(someRepository, "findById"),
  update: sinon.stub(someRepository, "update"),
  delete: sinon.stub(someRepository, "delete"),
});
```

Auth Controller Tests

Create `tests/unit/00-auth.test.js`. Use `afterEach(() => sinon.restore())` to reset all stubs between tests.

Your tests must cover:

- **Register - success:** the user does not already exist, returns **201**, and the response body does not include a **password** field.
 - **Register - duplicate user:** the repository indicates the user already exists, returns **409**.
 - **Login - success:** credentials are valid, returns **200** and the response body includes a **token** field.
 - **Login - invalid credentials:** the user is not found or the password does not match, returns **401**.
-

Category Controller Tests

Create `tests/unit/01-category.test.js`. Use `afterEach(() => sinon.restore())`. Import your shared helpers from `tests/mocks/category.mock.js`.

Your tests must cover all five CRUD operations:

- **Create** - repository resolves with a new category, returns **201**.
 - **Read all** - repository resolves with a non-empty array, returns **200**; repository resolves with an empty array, returns **404**.
 - **Read by ID** - repository resolves with a category, returns **200**; repository resolves with `null`, returns **404**.
 - **Update** - existing category is found and updated, returns **200**; category is not found, returns **404**.
 - **Delete** - existing category is found and deleted, returns **200**; category is not found, returns **404**.
-

Quiz Controller Tests

Create `tests/unit/02-quiz.test.js`. Use `afterEach(() => sinon.restore())`. Import your shared helpers from `tests/mocks/quiz.mock.js`.

Your tests must cover all five CRUD operations following the same pattern as the category tests above.

Note that a quiz belongs to a category. In your tests you do not need a real category - stub the repository so it behaves as if one exists.

JWT Middleware Tests

Create `tests/unit/03-jwtAuth.test.js`. Import the middleware function directly and call it with mock `req`, `res` and `next` objects. Use `sinon.stub()` for `next` so you can assert whether it was called.

Your tests must cover:

- No **Authorization** header is present - `next` is not called, returns **401**.
- **Authorization** header is present but does not begin with **Bearer** - returns **401**.
- A valid, correctly signed token is provided - `next` is called with no arguments and `req.user` is set to the decoded payload.
- An expired or tampered token is provided - `next` is not called, returns **401**.

For the valid token test, sign a real token using `jwt.sign()` with the same secret your middleware uses rather than mocking the JWT library itself.

RBAC Middleware Tests

Create `tests/unit/04-rbac.test.js`. Import the middleware directly and call it with mock `req`, `res` and `next` objects. Use `sinon.stub()` for `next`.

Your tests must cover:

- `req.user` is undefined - `next` is not called, returns `403`.
- The user's role is not in the permitted list - `next` is not called, returns `403`.
- The user's role is in the permitted list - `next` is called with no arguments.
- The middleware is configured with multiple permitted roles - a user with any matching role causes `next` to be called.

What to Expect in the Actual Practical Assessment

The actual assessment builds directly on everything covered in this practice. You should expect the following additional requirements:

Additional models and mock helpers. The full assessment includes three more models — `Question`, `GameSession`, and `Answer` — each with their own repository. You will need to create mock helper files for all five domains, not just `category` and `quiz`.

Question Controller Tests. You will write the same five CRUD tests as `category` and `quiz`, plus two additional cases for an OpenTDB import endpoint. This requires stubbing the global `fetch` function to simulate both a successful API response (`response_code: 0`) and a failed one (non-zero `response_code`), and asserting the correct status codes and response body shape in each case.

Game Session Controller Tests. You will test a `startSession` endpoint (valid quiz found → `201`; quiz not found → `404`) and a `finaliseSession` endpoint (score percentage correctly calculated → `200`; session not found or already finalised → appropriate error status).

Answer Controller Tests. You will test submitting an answer, including asserting that the response body indicates whether the submitted answer was correct, as well as handling a missing game session (`404`) and a missing question (`404`).

Code Coverage. The full assessment requires you to run `npm run test:coverage` and achieve at least 80% branch, statement, and function coverage across all controller and middleware files, as measured by `c8`. You will need a correctly configured `.c8rc` file.